PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

***Abstract*—We present PORTER (*Plan-Oriented Router for Teleportation-Enabled aRchitectures*), a router for multi-core distributed quantum computers that minimises inter-core EPR-pair consumption. PORTER unifies the three inter-core communication primitives used in prior work — state teleportation (TELEDATA), gate teleportation (TELEGATE), and multi-gate cat-entangler packets (TELEGATEPACKET) — under a single *plan-committed* decision loop: where TELESABRE and DSABRE score every inter-core candidate together with its staging SWAPs in one flat action pool, PORTER first scores whole remote plans on their physical and DAG-lookahead merits alone, commits to the best one, and only then generates the preparation SWAPs specific to that plan, so preparation effort is never spent on a plan that is not selected. A two-tier capacity-relief mechanism proactively routes qubits toward reservoir cores when candidate plans are blocked by insufficient free capacity, and a deterministic, seed-free *temporal locality-greedy* initial layout co-locates qubits by temporally discounted interaction weight in first-use order. On MQT-Bench suites at 25, 64, and 80 logical qubits, a *single* PORTER run from this single layout posts the lowest geometric-mean EPR count of the three routers on every suite – reducing EPR against TELESABRE by 54.3%, 49.8%, and 65.8%, and beating DSABRE's best-of-three protocol (43.1 vs. 47.0, 86.4 vs. 89.0, and 14.0 vs. 15.8) – while routing 11–61× faster per attempt than DSABRE. Routed from identical externally supplied layouts across thirty instances, PORTER also matches or beats DSABRE's routing stage (fifteen wins to twelve, three ties). PORTER is the only router of the three to complete every benchmark: on an 80-qubit, 23,381-CX circuit it finishes in 37 s where both baselines exceed a uniform 900 s compiler timeout. The implementation, an independent replay verifier, and the full comparison harness are publicly available.**

***Index Terms*—distributed quantum computing, circuit routing, qubit teleportation, plan-committed routing, EPR minimisation, qubit mapping, lookahead heuristic**

I. INTRODUCTION

Distributed quantum computers (DQCs) interconnect several small, high-fidelity quantum processing units (QPUs, or *cores*) with inter-core links that support remote entanglement generation [1], offering a near-term path to scale beyond what a single monolithic chip can host. Compiling a circuit onto a DQC decomposes into three coupled sub-problems [2]: *(i)* statically assigning logical qubits to cores, *(ii)* choosing, for each non-local two-qubit gate, which communication primitive realises it, and *(iii)* routing and scheduling the resulting sequence of intra-core SWAPs and inter-core operations subject to communication-port capacity. Every communication primitive consumes one or more EPR pairs, and EPR generation is slow, noisy, and rate-limited on current hardware [3], [4], making EPR minimisation the dominant compilation objective.

PORTER — a *Plan-Oriented Router for Teleportation-Enabled aRchitectures* — addresses stage (iii), ordering intra-core SWAPs and inter-core operations to minimise EPR consumption, and ships a deliberately lightweight answer to stage (i): a deterministic, seed-free initial layout built in microseconds, so that the full pipeline needs neither an external partitioner nor a per-circuit layout search (external layouts remain supported, and are used throughout the evaluation as controlled probes). The name is also a job description: like a hotel porter, the router's task is to decide which guests (logical qubits) to carry to which rooms (cores), when, and by which means — and to do so only after a plan for the whole errand has been settled. Unlike DSABRE [5], which restricts itself to state teleportation (TELEDATA), PORTER follows TELESABRE [6] in also supporting gate teleportation (TELEGATE) via the cat-entangler protocol [7], [8], and extends it with a same-control *packet* primitive (TELEGATEPACKET) that amortises one EPR pair over an ordered run of consecutive remote CX gates sharing a control qubit, in the spirit of gate-sequence amortisation used by AutoComm [9].

The router departs from TELESABRE and DSABRE in *when* it commits to core-preparation SWAPs. Both score inter-core candidates and their staging SWAPs together in one flat action pool. PORTER instead separates the decision into two phases: first, every legal remote *plan* — a teleport direction, a single telegate, a same-control packet, or a capacity-blocked variant of any of these — is scored using only its physical and DAG-lookahead properties, independent of how many preparation SWAPs it would take to make it physically legal right now; only *after* one plan is selected does the router generate and apply preparation SWAPs specific to that plan, repeating until the plan becomes physically ready or no further preparation SWAP improves it. This plan-first ordering means preparation effort is never spent scoring or partially staging a remote action that a better one will preempt.

Contributions.

- **A plan-first remote-action loop** that separates candidate scoring from preparation-SWAP generation (Section III-D), unifying teleport, telegate, and same-control packet primitives under one direct/latent/lookahead gain decomposition (Section III-E).
- **A two-tier capacity-relief mechanism**: a breadth-first search for the nearest reservoir core when a plan is capacity-blocked (Section III-H), plus an opportunistic comparator that lets a severely blocked candidate preempt a merely-good normal one. An ablation (Section IV-G) shows this mechanism is what makes the deterministic-greedy design complete at scale: without it, four of the six 80-qubit benchmarks deadlock.
- **A deterministic, seed-free initial layout** — temporal locality-greedy (Section III-A): each CX pair is weighted by temporally discounted interaction multiplicity, qubits

are visited in first-use order and co-located with their heaviest placed neighbour under a per-core free-slot reservation. A single layout built in microseconds replaces per-router layout search entirely, yet outperforms DSABRE's SabreLayout-based best-of-three protocol end-to-end on every suite we test.

- **An independent replay verifier** (Appendix D[1]) that re-executes the emitted operation stream against its own physical-layout simulation and checks gate coverage, DAG-order faithfulness, and every physical legality constraint (coupling-edge adjacency, port occupancy, cat-section isolation), used to certify correctness across 60 randomized circuits and the full evaluation suite — and itself validated by fault injection: ten classes of single-point stream corruption, every mutant rejected by the intended check.
- **A head-to-head evaluation** against TELESABRE and DSABRE on identical physical topologies at 25, 64, and 80 logical qubits (Section IV), under each router's own initial-layout protocol and under shared fixed layouts (with an external `pytket-dqc`-generated layout source in Appendix B).

The PORTER implementation, the replay verifier, the full cross-router comparison harness, and the raw per-circuit results are publicly available at https://github.com/ebony72/porter.

II. Background

A. Quantum Circuits and Routing

A quantum circuit acts on n logical qubits through a sequence of gates; we work in the restricted basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ used by MQT-Bench's Qiskit-opt3-transpiled circuits, i.e. arbitrary one-qubit (1Q) gates plus the two-qubit (2Q) CX gate. A circuit is represented as a directed acyclic graph (DAG): for every logical qubit, consecutive gates acting on it are connected by a dependency edge. The *front* (or *DAG front*) is the set of gates with no unexecuted predecessor; only front gates can execute or be routed towards execution at a given step. Figure 1 shows a five-gate circuit and its DAG; note that gates sharing a qubit (e.g. two CX gates with the same control) are chained by a dependency edge and therefore *never* appear in the front simultaneously — a fact that matters for packet collection (Section III-F).

Physical qubits are partitioned into *cores*; each core has an intra-core coupling graph, and pairs of cores may be connected by an inter-core link whose two endpoints are designated *communication ports* (Figure 2). A *layout* maps every logical qubit to a physical position. A 2Q gate is *local* if both operands share a core and *remote* otherwise. Routing a local gate that is not yet adjacent requires intra-core SWAPs; routing a remote gate requires an inter-core communication primitive.

B. Communication Primitives

Three inter-core primitives are in play, sketched in Figure 3; each consumes exactly one EPR pair.

[1]Appendices A–D are provided in the separate supplementary-material document (`sfc_appendix.pdf`).



Fig. 1. (a) A five-CX circuit on four logical qubits and (b) its dependency DAG (edges labelled by the shared qubit). The DAG front — the gates with no unexecuted predecessor — is shaded: initially $\{g_1, g_2\}$; once both execute it becomes $\{g_3\}$, then $\{g_4, g_5\}$.

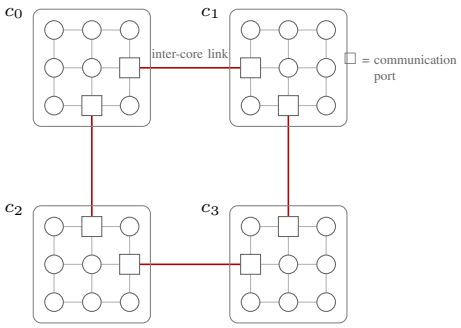


Fig. 2. Architecture A: a 2×2 array of cores, each a 3×3 grid of physical qubits (circles) with nearest-neighbour intra-core coupling (grey edges). Squares are communication ports; every inter-core link joins one port in each of two adjacent cores and is the only place EPR pairs are consumed. Architecture H (2×3 array of 4×4 cores, Section IV) follows the same pattern.

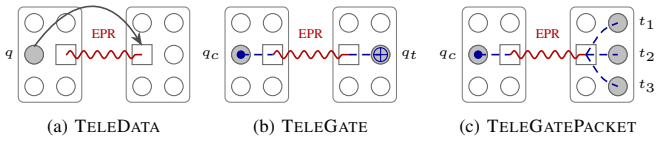
State teleportation (TELEDATA). Consumes one EPR pair to move a logical qubit's state from a physical position adjacent to a free source-side communication port to the (necessarily free) destination-side port in a neighbouring core [10]. The qubit then participates in local gates in its new core.

Gate teleportation (TELEGATE). Executes a single remote CX directly across two adjacent cores via a cat-entangler protocol [7], [8]: one EPR pair entangles the two communication ports, the remote CX is applied using the target-side port as a proxy for the control qubit, and the entanglement is consumed on disentangling. Neither logical qubit physically moves.

Same-control packets (TELEGATEPACKET). When several consecutive remote CX gates in the DAG front share the same control qubit and land in the same target core, PORTER amortises them onto a *single* cat-entangler resource: the entangling and disentangling steps bracket an ordered sequence of remote CX and target-core 1Q gates, all executed while the two communication ports stay reserved. This is the same amortisation idea used by AutoComm [9], specialised to the shared-control case that arises naturally at the DAG front (Section II-A).

C. SABRE

SABRE [11] routes a single-core circuit by repeatedly selecting the SWAP that minimises a front-layer distance term plus a weighted extended-set lookahead term, both expressed as Δ (old distance minus new distance) on gates sharing a qubit with the candidate SWAP. PORTER inherits this scoring



(a) TELEDATA (b) TELEGATE (c) TELEGATEPACKET

Fig. 3. The three communication primitives; each consumes one EPR pair between the two communication ports (squares). (a) TELEDATA moves qubit q (which must sit adjacent to its free port) onto the free destination port. (b) TELEGATE executes one remote CX through a cat-entangler: the target-side port acts as a proxy for the control q_c; neither qubit moves. (c) TELEGATEPACKET brackets an ordered run of remote CX gates sharing the control q_c between one cat-entangler/disentangler pair, amortising a single EPR pair over the whole run; each target is routed next to the port in turn while the ports stay reserved.

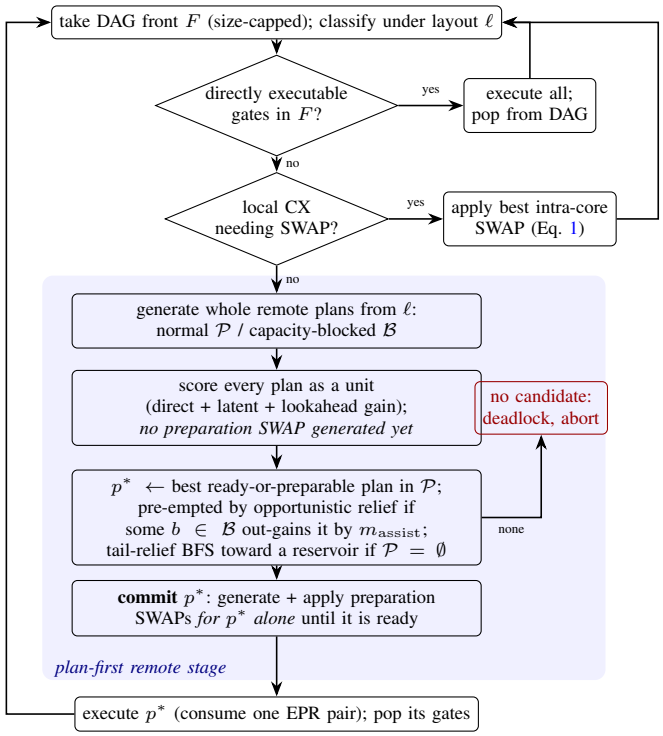


Fig. 4. One iteration of PORTER's compile loop (Algorithm 1). Local work always drains first; the shaded plan-first remote stage scores whole remote plans before any preparation SWAP exists, commits to one, and only then stages it.

form for its intra-core stage (Section III-C) and extends the same direct/lookahead decomposition to its inter-core plan scorer (Section III-E).

III. THE PORTER ROUTER

Given a circuit and an initial layout ℓ_0 (which PORTER builds itself, Section III-A), PORTER repeats the following loop (Algorithm 1) until every gate has been emitted or a configurable iteration budget is exhausted. Figure 4 charts one iteration; its shaded region is the plan-first remote stage that distinguishes PORTER from flat-pool routers, and Section III-G walks through it on a concrete example.

A. Initial Layout

PORTER constructs its own starting layout ℓ_0 with a single deterministic, seed-free heuristic and does not require an external partitioner or a best-of-N search. It respects the capacity invariant the routing loop depends on: every core keeps at least $f_{\min}$ free physical positions ($f_{\min}$=2), so that from the first iteration any core can serve as a TELEDATA destination and as a reservoir for capacity relief (Section III-H). A logical qubit is therefore never placed into a core that already holds $|c| - f_{\min}$ logicals, where $|c|$ is the core's physical capacity.

Algorithm 1 PORTER `compile`(C, ℓ_0)

$\ell \leftarrow \ell_0$; initialise DAG active/frontier sets from C
while DAG has active gates **do**
  $F \leftarrow$ DAG front (bounded by `dag_front_limit`)
  classify F into singles, direct-local CX, local-CX-needing-SWAP, adjacent-remote CX, non-adjacent-remote CX *(Sec. III-B)*
  if singles or direct-local CX present **then**
    execute them all; remove from DAG; **continue**
  end if
  if local-CX-needing-SWAP present **then**
    generate candidate intra-core SWAPs, apply the best-scoring one (Eq. 1); **continue** *(Sec. III-C)*
  end if
  generate normal remote plans $\mathcal{P}$ and blocked-teleport plans $\mathcal{B}$ *(Sec. III-E)*
  if $\mathcal{P} \neq \emptyset$ **then**
    $p^* \leftarrow$ best ready-or-preparable plan in $\mathcal{P}$
    if some $b \in \mathcal{B}$ has gain $\gg$ gain(p^*) **then**
      $p^* \leftarrow$ best opportunistic capacity-relief plan for b *(Sec. III-H)*
    end if
  else
    BFS from each blocked target core to its nearest reservoir; $p^* \leftarrow$ best tail-relief plan *(Sec. III-H)*
  end if
  if p^* is `None` **then**
    report deadlock; abort
  end if
  apply p^*'s own preparation SWAPs until ready, then execute it
end while
return emitted operations, final layout, stats

Temporal locality-greedy. The heuristic runs in three deterministic steps. *(i) Weighting.* Each CX gate $g = (q_i, q_j)$ contributes $\gamma^{\ell(g)}$ to the interaction weight $w(q_i, q_j)$ of its qubit pair, where $\ell(g)$ is g's as-soon-as-possible layer in the CX-only DAG and γ=0.99; early interactions thus weigh more than late ones, while repeated interactions on a pair accumulate. *(ii) Core assignment.* The logical qubits are visited in the order they first appear as a CX operand and each is placed in the core holding its heaviest already-placed neighbour when that core still has headroom above $f_{\min}$, falling back to the core with the most remaining reserved capacity otherwise. *(iii) Physical placement.* Within each core, the assigned logicals are laid onto physical positions by descending total interaction weight onto ascending intra-core centrality (sum of local distances), so

the most-connected qubits take the most central slots; unused slots are left idle.

This single construction subsumes what the earlier design split across two heuristics and a best-of-six search. The temporal discount front-loads the opening layer of interactions, recovering the first-CX-layer co-location a "balanced first-layer" scan gave; the accumulated multiplicity together with the first-appearance visit order tracks chain-structured circuits a first-layer scan cannot – a GHZ-style chain exposes only one disjoint pair in its first CX layer, yet successive chain neighbours share weight and are visited in sequence, so they land in the same core. Of the three steps the visit order carries the most weight: replacing the first-appearance order with a random permutation inflates geometric-mean EPR from 55.4 to 103.3 over the eighteen benchmark instances (Section IV), whereas replacing centrality placement with a random within-core shuffle costs only $55.4 \rightarrow 56.4$.

External layouts. When ℓ_0 instead comes from an outside mapper that may fill a core to capacity (Appendix B), PORTER first repairs it (`repair_full_cores`): it evicts one logical qubit from every saturated core into the core with the most spare capacity, restoring the $f_{\min}$ reservation before routing. A core packed to its physical capacity can never clear an occupied communication port with local SWAPs, so without this step its qubits would be unreachable by any teleport or telegate. PORTER's own heuristic reserves $f_{\min}$ by construction and never invokes the repair.

B. Front Classification

Every gate g in the (size-capped) DAG front is classified under the current layout ℓ: 1Q gates are always directly executable; a CX gate $g = (q_i, q_j)$ with $\kappa_\ell(q_i) = \kappa_\ell(q_j) = c$ (same core) is *direct-local* if $d_{\mathrm{L}}(c, \ell(q_i), \ell(q_j)) = 1$ and *local-needing-SWAP* otherwise; a CX gate with $\kappa_\ell(q_i) \neq \kappa_\ell(q_j)$ is *adjacent-remote* if the two cores are core-graph neighbours and *non-adjacent-remote* otherwise. Directly executable gates are always drained first and completely, so remote-stage decisions are only made once the front contains solely gates that need routing.

C. Intra-Core SWAP Scoring

For a local-needing-SWAP gate set F_{loc}, PORTER builds candidate SWAP edges from the coupling-graph neighbours of every physical position touched by F_{loc}, simulates each SWAP, and scores it as

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}}\, \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

the mean post-SWAP local distance over F_{loc} plus a weighted mean over an extended set $\mathcal{E}_{\mathrm{loc}}$: up to N_{loc}=20 local CX gates collected from the next L_{loc}=12 DAG layers via a virtual frontier simulation (Section IV-A). Ties are broken first by the raw front-layer distance, then, if still tied, by a decision uniformly sampled from a seeded random-number generator, with every seeded tie-break recorded in `RouterStats` for reproducibility auditing.

D. Plan-First Remote Stage

PORTER manipulates remote work as *plans*, and the word is chosen deliberately. A plan is a complete, committable specification of one inter-core action: which primitive fires (TeleData, TeleGate, or TeleGatePacket), which gate or same-control gate run it resolves, its direction (which endpoint moves, or which side hosts the control), and consequently its *readiness conditions* — which logical endpoint must sit adjacent to which communication port, and which ports must be free. What a plan deliberately leaves unspecified is the SWAP sequence that will establish those conditions. This is the classical deliberation-versus-execution split: alternatives are compared at the level of *what* remote work to do next, each scored as if its preconditions were met, and only the one committed plan is handed to the preparation machinery that works out *how*. A flat action pool, by contrast, makes the same choice implicitly and one micro-move at a time: every staging SWAP it selects is also a vote for whichever remote action that SWAP happens to serve, votes for competing actions interleave, and staging effort is routinely spent on actions that are never taken (Section III-G gives a concrete front where this changes the outcome).

When the front contains only remote CX gates, PORTER generates two disjoint candidate sets from the current layout:

- **Normal plans** $\mathcal{P}$: for every adjacent-remote CX gate, a single TeleGate; if a same-control run of ≥ 2 remote CX gates can be collected by simulating the DAG front forward (Section III-F), a TeleGatePacket; and, for either endpoint, a TeleData plan into the partner's core *if* that core already has at least `min_free_before_teleport` ($f_{\min}$=2) free physical positions. For non-adjacent-remote CX gates, only distance-reducing TeleData hops (one core-graph step closer to the gate's other endpoint) with sufficient destination capacity are generated.
- **Blocked plans** $\mathcal{B}$: the same directional TeleData candidates, but for destination cores that currently fail the $f_{\min}$ capacity test. These are never selected directly; they exist to detect when *no* normal plan exists because every candidate destination is saturated, or when a normal plan is technically available but a severely blocked plan promises much higher gain if capacity is relieved first.

Each plan is scored (Section III-E) *before* any preparation SWAP for it is generated. The router then:

1) selects the best plan in $\mathcal{P}$ that is either already physically ready or admits at least one improving preparation SWAP right now (see the preparation rule below);
2) compares its gain against the best plan in $\mathcal{B}$; if the blocked plan's gain exceeds the selected normal plan's by more than a fixed margin m_{assist}=5.0, the router instead searches for an *opportunistic capacity-relief* plan for the blocked plan's saturated destination core (Section III-H) and, if one exists, executes that instead;
3) if $\mathcal{P} = \emptyset$ (every remote CX gate is non-adjacent), the router runs the same breadth-first capacity-relief search directly on $\mathcal{B}$'s destination cores, grouped by hop count, taking the shortest-hop group with a viable candidate.

Once a plan is committed, the router repeatedly generates and applies *only that plan's* preparation SWAPs until it becomes ready, no further improving preparation SWAP exists, or the full decision state (plan identity, layout, RNG state) repeats — the latter raising a hard error rather than looping forever, bounded by `max_prep_iterations_per_plan` (10^4).

Preparation SWAPs. A committed plan's readiness requirements are exactly two *(core, neighbour-core, endpoint)* triples (source and destination side for TELEDATA; control and target side for TELEGATE/TELEGATEPACKET, whose target-side routing is deferred to execution time, Section III-F). For each unmet requirement, candidate preparation SWAPs are restricted to edges incident to either the plan's logical endpoint or a nearest free physical position tied for minimum distance to the relevant communication port (only while that port remains occupied); each candidate is scored by the post-SWAP value of the same readiness-cost function used to rank blocked plans (Section III-E), and at most one improving SWAP per core side is applied per pass.

E. Plan Scoring: Direct, Latent, and Lookahead Gain

Every plan's score is $\mathrm{total} = \mathrm{direct} + \mathrm{lookahead}$, computed differently per plan kind but sharing one Δ-form convention: for a CX gate $g = (q_i, q_j)$ and a candidate qubit relocation, $\Delta(g) = d_C(\kappa(q_i), \kappa(q_j)) - d_C(\kappa'(q_i), \kappa'(q_j))$, the reduction in core-graph distance between g's two cores.

TELEGATE contributes a constant direct gain of 1 (one remote CX resolved) and no lookahead term.

TELEGATEPACKET contributes a direct gain equal to the number of remote CX gates in the collected packet – every gate in an accepted packet is current, not speculative, work, since the whole packet executes under one cat-entangler resource once started.

TELEDATA decomposes its influence into three tiers, all computed by simulating the DAG frontier under the hypothetical post-teleport core assignment (Section IV-A):

- *Direct*: front-layer CX gates already involving the moving qubit, plus the seed gate itself.
- *Latent-direct*: CX gates on the moving qubit that appear in the simulated DAG front while the simulator can still make virtual progress with only 1Q gates and now-local CX gates (i.e. gates whose post-teleport cores coincide); these gates are current pressure, not prediction, because the candidate teleport directly determines their post-move core distance the moment they surface.
- *Lookahead*: CX gates on the moving qubit found in up to L_{tp}=16 further simulated DAG layers (capped at N_{tp}=50 gates) beyond the point where the latent-direct simulation naturally stops (no further gate is virtually executable). Layer k's contribution is weighted by $\max(w_{\min}, w_0\gamma^{k-1})$ with w_0=0.9, γ=0.9, $w_{\min}$=0.5.

$$\mathrm{direct} = \sum_{g\in\mathrm{direct}} \Delta(g) + w_{\mathrm{ld}} \sum_{g\in\mathrm{latent}} \Delta(g), \tag{2}$$

$$\mathrm{lookahead} = \sum_{(g,k)\in\mathrm{la}} w_k\,\Delta(g), \tag{3}$$

TABLE I
REMOTE PLANS GENERATED FOR THE FRONT OF FIGURE 5, WITH TOTAL GAINS (SECTION III-E).

Plan	Set	Gain	Readiness	Outcome
TELEGATEPACKET(q_0; $g_1 g_2 g_3$)	$\mathcal{P}$	3.0	1 prep SWAP	**committed**
TELEGATE(g_4)	$\mathcal{P}$	1.0	ready now	dominated
TELEGATE(g_1)	$\mathcal{P}$	1.0	1 prep SWAP	dominated
TELEDATA($q_1 \to c_0$)	$\mathcal{P}$	1.0	prep needed	dominated
TELEDATA($q_5 \to c_0$)	$\mathcal{P}$	1.0	prep needed	committed later
TELEDATA($q_0 \to c_1$)	$\mathcal{B}$	$\approx$2.1	blocked	margin $< m_{\mathrm{assist}}$
TELEDATA($q_4 \to c_1$)	$\mathcal{B}$	1.0	blocked	dominated

with w_{ld}=1.0 (`teleport_latent_direct_weight`). A capacity-risk term $1/\max(1, f_{\mathrm{dst}}-1)$ (post-move free count in the destination core) is tracked alongside the score for use as a tie-break dimension but does not enter total directly; plans are primarily ranked by total gain (descending), then readiness, then preparation cost, then plan kind (TELEDATA $\succ$ TELEGATEPACKET $\succ$ TELEGATE), then capacity risk, then lookahead gain, with a fully deterministic stable key as the final tie-break before any seeded random choice.

F. Same-Control Packet Collection

Starting from a seed adjacent-remote CX gate $g = (q_c, q_t)$ with control core c_c and target core c_t, PORTER simulates the DAG frontier forward for up to `extended_packet_window` (120) layers, greedily absorbing into the packet: (i) any front CX gate sharing control q_c whose target is also in c_t, up to `max_packet_size` (8) such gates; and (ii) any front 1Q gate whose qubit is already in c_t, since these execute freely while the packet's cat resource is live. Collection stops at the first layer that contributes nothing, or once the size cap is reached. Because this DAG-frontier simulation admits gates from later generations that are not literally in the (bounded) current DAG front, a packet may contain gates unreachable by a naively front-limited scan; the collected order is preserved and replayed exactly at commit time, as one `cat_entangler` (consuming the packet's single EPR pair), one `cat_cx` per member — each target routed to its communication port by ordinary local SWAPs, forbidden from touching the two live ports — and a closing `cat_disentangler`.

G. Running Example: Why Commit to a Plan?

Figure 5 shows a front where plan commitment, the packet vocabulary, and the $f_{\min}$ capacity gate all interact, and where PORTER's decision differs from a flat action pool's. Two adjacent cores of architecture A are shown. Core c_0 holds q_0, q_4 (adjacent to the port), and q_6, with six free positions; core c_1 holds eight qubits — the targets q_1, q_2, q_3, the port-adjacent q_5, and four spectators — so its *only* free position is its port. The remaining circuit fragment is g_1=CX(q_0, q_1), g_2=CX(q_0, q_2), g_3=CX(q_0, q_3), g_5=CX(q_0, q_6), chained on q_0, plus an independent g_4=CX(q_4, q_5). The front is therefore $\{g_1, g_4\}$ — g_2 and g_3 share q_0 with g_1 and only surface after it executes. All front gates are adjacent-remote, so the plan-first stage fires.

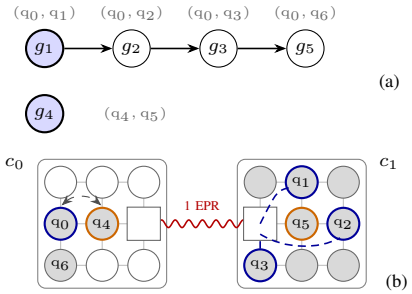


Fig. 5. Running example. (a) Remaining circuit fragment as a DAG; the front (shaded) is $\{g_1, g_4\}$. (b) Layout on two adjacent cores of architecture A. Blue circles are the committed TELEGATEPACKET's participants (q_0 and the targets q_1, q_2, q_3, reached by cat-CXs, dashed); orange circles are the endpoints of the competing, already-ready single TELEGATE for g_4; grey circles are spectators. Core c_1's only free position is its port, so $\text{free}(c_1) = 1 < f_{\min}$ and every TELEDATA into c_1 is capacity-blocked. The dashed double arrow is the packet's one preparation SWAP, which moves q_0 next to the port.

What PORTER does. Table I lists the generated plans. Seeding packet collection at g_1, the frontier simulation absorbs g_2 and g_3 (same control q_0, targets in c_1): a three-gate packet with direct gain 3. Both single telegates and all TELEDATA plans into c_0 gain ≈1. TELEDATA($q_0 \to c_1$) would gain more — direct 1 for g_1, latent-direct 2 for g_2, g_3, minus 0.9 of lookahead for g_5, whose partner q_6 stays in c_0 — but c_1 fails the $f_{\min}$ test, so it lands in $\mathcal{B}$; its gain ≈2.1 does not exceed the packet's 3.0 by m_{assist}=5, so no capacity relief pre-empts. Plans are ranked by gain first — readiness is only a tie-break — so the *unready* packet (gain 3) beats the *ready* telegate for g_4 (gain 1). PORTER commits the packet, generates its one preparation SWAP (q_0 next to the port), then emits one cat-entangler, three cat-CXs (each target routed beside the port in turn), and the disentangler: *one* EPR pair for g_1, g_2, g_3. The packet's completion exposes g_4 and g_5 together; the locality-first stage drains the now-local g_5 first (one SWAP plus a local CX), and the remote stage then re-generates plans for g_4: the single telegate and TELEDATA($q_5 \to c_0$) now tie at gain 1, and the plan-kind tie-break (TELEDATA ≻ TELEGATE, Section III-E) commits the teleport — q_5 lands on c_0's port and g_4 finishes as a local CX. Two EPR pairs for the whole fragment. This is not a stylised trace: it is exactly the operation stream the implementation emits on this instance, replayed and certified by the independent verifier of Appendix D, which includes the full operation-by-operation certified trace.

What a flat pool does. A router that scores individual actions — SWAPs and inter-core operations in one pool — fires the zero-cost ready telegate for g_4 first. The deeper problem is the staging SWAP the packet needs: exchanging q_0 and q_4 moves g_1's control one step closer to the port while moving g_4's one step away, so under a front-averaged distance metric of the kind used by SABRE descendants its score is ≈0; the information that makes this SWAP worth applying — three remote CXs behind g_1 amortised onto one EPR pair — is a property of a *whole plan*, invisible at single-action granularity. Vocabulary compounds the gap: g_2 and g_3 enter the front one at a time, so TELESABRE pays one telegate each, four EPR pairs for the fragment; DSABRE, restricted to TELEDATA, cannot move q_0 into the saturated c_1 without first evicting a resident (its `_force_make_room`, one more teleport), and ferrying q_1, q_2, q_3 out to c_0 instead costs three EPR pairs before g_4 is even addressed. Commitment finally protects the choice once made: only the packet's preparation SWAPs are generated — no staging effort is spent on the six losing plans — and the committed plan cannot be pre-empted mid-preparation by a candidate that its own staging transiently improved, a churn a flat pool permits by construction.

H. Capacity Relief

Tail-relief BFS. When every remote plan is capacity-blocked, PORTER runs a breadth-first search from each blocked destination core to the nearest *reservoir* core – one with at least $f_{\min}$ free slots – optionally chained beyond one hop (`capacity_relief_chain_enabled`). Every logical qubit currently in the BFS path's penultimate core becomes a tail-relief TELEDATA candidate into the reservoir, scored and selected by the same plan machinery, grouped and tried in ascending hop-count order.

Opportunistic relief. Even when a normal plan is available, a severely blocked plan can still win if relieving its target core's capacity would unlock substantially more gain (Section III-D); this differs from tail-relief BFS in firing pre-emptively, before any plan is actually starved, whenever the margin m_{assist} is exceeded.

I. Computational Complexity

Let N be the circuit's gate count, n its logical-qubit count, K the number of cores, and M the per-core physical-qubit count ($P{=}KM$ total physical qubits). Hardware distance tables (d_{L} per-core, d_{C} core-graph) are precomputed once at construction, in $O(KM^2 \log M + K^2 \log K)$, matching the standard two-level Dijkstra decomposition used by comparable routers [5]. Per iteration, front classification and direct-execution draining cost $O(|F|)$; intra-core SWAP scoring evaluates $O(M)$ candidate edges each at $O(|F_{\text{loc}}| + N_{\text{loc}})$ via one $O(|\text{frontier}|)$-cost DAG snapshot, reused across candidates; remote-plan scoring evaluates $O(n)$ candidates each at $O(N_{\text{tp}})$ via the same amortised snapshot cost. Free-slot-queue maintenance is $O(1)$ amortised per layout mutation since only the touched core(s) are refreshed (Section IV-A). Summing over N iterations gives $T_{\text{total}} = O\big(N(Mn + KM^2 \log M)\big)$ excluding the one-time precomputation term, in line with the per-iteration cost of other core-local distributed routers [5], [6].

IV. EXPERIMENTAL EVALUATION

A. Implementation and Verification

PORTER is implemented in Python and is publicly available, together with the verifier and comparison harness used below (Section I). The implementation is fully deterministic: for a fixed (circuit, hardware, random seed) triple the emitted operation stream is byte-identical across runs, and every seeded tie-break is counted in the emitted statistics, so behaviour preservation across refactorings can be certified

by hashing the serialized operation stream. Compile time is kept low by incremental data structures — chiefly the `DagFrontierSimulator` shared by extended-set collection and teleport lookahead (Sections III-C–III-F and III-E), a copy-on-write view over the live DAG state that tracks a private overlay of predecessor-count decrements against the shared `active`/`pred_base` maps, so that a snapshot costs $O(|\text{frontier}|)$ rather than $O(|\text{remaining circuit}|)$ and several independent virtual simulations can run from the same live DAG state without interfering — together with incremental DAG-frontier maintenance, cached hardware distance tables, $O(1)$ free-slot accounting, and apply/restore SWAP scoring in place of layout cloning. These reduce wall-clock compile time by roughly $4\times$ relative to a straightforward implementation of the same algorithm, with byte-identical output; Section IV-H reports absolute compile times.

Every reported run is certified by an independent replay verifier that treats the router as a black box: from the emitted operation stream, the original circuit, and the initial layout alone it re-derives gate coverage (each gate exactly once), per-qubit dependency order, coupling-edge legality, the port and isolation rules of all three communication primitives, and consistency of the emitted statistics. The verifier passes on 60 randomized circuits and on every instance in the evaluation below, and is itself validated by fault injection; Appendix D details its checks, a certified operation-by-operation trace of the running example, and the fault-injection study.

B. Setup

Architectures. We use the same two grid-of-grids topologies introduced by TELESABRE [6] and re-used by DSABRE [5]: architecture A, a 2×2 array of 3×3 cores (36 physical qubits, 4 inter-core links), and architecture H, a 2×3 array of 4×4 cores (96 physical qubits, 7 inter-core links). Both are instantiated identically for all three routers.

Benchmarks. We use the same MQT-Bench [12] circuit files DSABRE's own evaluation uses (confirmed by exact CX-count match against its published tables): a 25-qubit suite (AE, GHZ, Graphstate, QFT, QNN, Random) on architecture A, and a 64-qubit suite (the same six families) on architecture H, with QNN using the deeper 8,126-CX variant. To test at intermediate scale on the same architecture H we add an 80-qubit suite of the same six families, generated with `mqt.bench` 2.2.2. The 80-qubit AE and QNN instances cannot be produced by the stock generator and are reconstructed to match its own construction exactly; in particular, QNN is the deep 12,718-CX ZZ-feature-map circuit rather than the degenerate 79-CX chain the current generator emits (whose interaction graph is identical to GHZ's). Appendix A details both reconstructions and their byte-level validation. Table II lists CX counts.

Baselines. We compare against TELESABRE [6] and DSABRE [5]. For DSABRE we report only `dSABRE_BurstExt`, its BFS-layer extended-set variant and the project's own default/production router (its topological-extended-set ablation, DSABRE-Topo, is omitted). TELESABRE's stock reference C binary crashes deterministically on architecture A, whose 9-slot cores are materially smaller than the 4×4 cores it was originally evaluated on: its Hungarian initial-layout pass can leave a logical qubit unassigned when no core's remaining capacity exceeds its free-slot threshold, and the unassigned sentinel is then used as a negative array index. All TELESABRE numbers in this paper use a minimally patched binary that falls back to the least-loaded core in that case; the patch and a detailed account of the issue are provided in our public repository.

TABLE II
BENCHMARK CIRCUIT CX COUNTS (MEASUREMENTS/BARRIERS STRIPPED).

Circuit	25q (arch. A)	64q (arch. H)	80q (arch. H)
AE	558	1962	2780
GHZ	24	63	79
Graphstate	25	64	80
QFT	580	1966	2660
QNN	1223	8126	12718
Random	1124	1627	23381

Compiler timeout. Every individual compiler invocation – one TELESABRE subprocess run, one DSABRE routing-pass call, or one PORTER `compile` call – is capped at 900 s wall-clock; a call that exceeds it is recorded the same way as any other failure to complete (missing entry, excluded from that protocol's geometric mean). This is a harness-level bound applied uniformly across all three routers and all three protocols below, not specific to any one circuit.

C. Initial-Layout Protocols

We evaluate under three initial-layout protocols, designed to separate routing quality from initial-layout quality.

Own-protocol comparison. Each router is first evaluated under its own established best-practice initial-layout protocol. For the two baselines this is best-of-three, matching the convention already used to compare TELESABRE and DSABRE to each other [5]; for PORTER it is a single deterministic run:

- **TELESABRE**: the compiled C binary's built-in hungarian initial-layout heuristic, config seeds 0–2, best EPR count of the successful runs.
- **DSABRE**: Qiskit `SabreLayout` on the corners-removed architecture graph (seeds 0–2, with the two least useful corners of *every* core removed — see below) followed by SABRE-style forward → backward → forward routing passes, best EPR count.
- **PORTER**: a *single* routing pass from its single deterministic temporal locality-greedy layout (Section III-A), router seed 0. PORTER performs no layout search and no seed sweep, so its column reports one attempt per circuit where each baseline's reports the best of three; every number is exactly reproducible from the circuit file alone.

DSABRE's corner removal reserves low-degree physical qubits as communication slots by excluding them from the SabreLayout coupling map. Its published selection takes the four lowest-degree vertices of the *whole-chip* graph; on both of our architectures, whose physical-qubit ids are assigned

TABLE III
HARDWARE COST AND HEURISTIC PARAMETERS (SEE SECTION III).

Parameter	Symbol	Value
Min. free before TELEDATA	$f_{\min}$	2
Opportunistic-relief margin	m_{assist}	5.0
Local-SWAP extended-set weight	λ_{loc}	0.8
Local extended-set max layers	L_{loc}	12
Local extended-set max CX	N_{loc}	20
Packet collection window (layers)	–	120
Max packet size (remote CX)	–	8
TeleData lookahead max layers	L_{tp}	16
TeleData lookahead max CX	N_{tp}	50
Lookahead weight / decay / floor	$w_0, \gamma, w_{\min}$	0.9, 0.9, 0.5
Latent-direct weight	w_{ld}	1.0

core-by-core, these four vertices are, by construction, the four corners of a single core (core 0 in our numbering), so the remaining cores receive no reserved slots at all and regularly start fully packed. We strengthen the baseline's protocol here: we instead remove, in *every* core, the two corners farthest (by total shortest-path distance in the chip graph) from all other physical qubits — the least useful positions, whose reservation costs the mapper the least. Two per core is also the natural budget: removing all four corners of every core would leave architecture A only $4 \times (9 - 4) = 20$ usable slots, too few for its 25-qubit circuits, whereas two leave $4 \times (9 - 2) = 28$ there and $6 \times (16 - 2) = 84$ on architecture H, enough for the 80-qubit suite. We still do not reuse these SabreLayout candidates for PORTER in this comparison, to keep each router's own-protocol result tied to its own established initial-layout heuristic.

Shared-layout comparison. To isolate routing quality from initial-layout quality, we additionally route both DSABRE and PORTER from TELESABRE's *own* initial layout: for the winning seed of the run above, we extract, from TELESABRE's optional JSON trace, the physical-to-logical mapping recorded at iteration 0 – i.e. the layout TELESABRE itself settled on after its internal forward/backward/forward layout refinement, immediately before its accepted forward routing pass – and feed that identical mapping, unmodified, to a single DSABRE run and a single PORTER run (no per-router layout search; the layout is now fixed and given). TELESABRE's hungarian heuristic reserves free-slot headroom per core by construction (tuned to its own teleport threshold), so it is compatible with PORTER's teleport precondition without adjustment on every circuit we tested.

A third, independent layout source — a placement generated by `pytket-dqc`'s `HypergraphPartitioning` allocator [13], routed by both DSABRE and PORTER — is reported and discussed in Appendix B.

D. Own-Protocol Results

TELESABRE's EPR count is teledata plus telegate operations; DSABRE's is teledata only; PORTER's is teleport plus telegate plus telegate-packet operations (one EPR pair per packet, regardless of how many remote CX gates it amortises). All three report in the same one-EPR-pair unit. SWAP counts intra-core SWAPs only.

TABLE IV
25-QUBIT SUITE, OWN-PROTOCOL COMPARISON (ARCHITECTURE A, 2×2 CORES OF 3×3, 36 PHYSICAL QUBITS). TELESABRE AND DSABRE ARE BEST-OF-3 SEEDS; PORTER IS A *single* RUN FROM ITS SINGLE DETERMINISTIC LAYOUT (SECTION IV-C). Δ_{TS} IS PORTER'S EPR CHANGE RELATIVE TO TELESABRE; NEGATIVE IS BETTER.

	TELESABRE		DSABRE		PORTER		
Circuit	EPR	SWAP	EPR	SWAP	EPR	SWAP	Δ_{TS}
ae	118	370	68	395	65	328	−44.9
ghz	11	30	4	17	4	29	−63.6
graphstate	17	43	7	24	7	34	−58.8
qft	141	371	89	427	74	327	−47.5
qnn	357	770	153	761	119	642	−66.7
random	632	1512	417	1438	401	1327	−36.6
gmean (6)	94.3		47.0		**43.1**		**−54.3**

TABLE V
64-QUBIT SUITE, OWN-PROTOCOL COMPARISON (ARCHITECTURE H, 2×3 CORES OF 4×4, 96 PHYSICAL QUBITS). METHODOLOGY AND NOTATION AS TABLE IV. *TELESABRE FAILS TO CONVERGE WITHIN 3 SEEDS; GMEAN IS OVER THE 5 CIRCUITS ON WHICH IT DOES.

	TELESABRE		DSABRE		PORTER		
Circuit	EPR	SWAP	EPR	SWAP	EPR	SWAP	Δ_{TS}
ae	323	1508	262	2016	177	1627	−45.2
ghz	11	111	9	73	6	86	−45.5
graphstate	76	379	18	103	20	131	−73.7
qft	410	1872	258	2295	262	1846	−36.1
qnn	1365	6292	509	8610	864	6595	−36.7
random*	—	—	770	4077	737	4048	—
gmean (5)	172.1		89.0		**86.4**		**−49.8**

Across the three suites the own-protocol comparison supports one headline claim: *a single* PORTER *run from its single deterministic layout posts the lowest geometric-mean EPR count of the three routers on every suite*, despite each baseline reporting the best of three attempts – 43.1 vs. DSABRE's 47.0 and TELESABRE's 94.3 at 25 qubits, 86.4 vs. 89.0 and 172.1 at 64, and 14.0 vs. 15.8 and 40.9 at 80 (the 80-qubit gmeans are over the two circuits TELESABRE completes; the fuller circuit-by-circuit picture, including six instances on which DSABRE wins, is below). It is also the only router to complete all eighteen instances (DSABRE completes seventeen, TELESABRE thirteen). The per-suite detail follows.

25-qubit suite. PORTER reduces geometric-mean EPR consumption by **54.3%** against TELESABRE, with per-circuit reductions from −36.6% (Random) to −66.7% (QNN) and no regressions. Against DSABRE it wins AE, QFT, QNN, and Random outright and ties GHZ (both 4) and Graphstate (both 7), conceding nothing at this scale. On the CX-dense circuits the margin is clearest (QNN: 119 vs. 153; QFT: 74 vs. 89), consistent with PORTER's TELEGATEPACKET primitive amortising runs of shared-control remote CX gates that a teledata-only router must pay for individually (Section IV-G isolates this effect).

64-qubit suite. TELESABRE fails to converge within three seeds only on Random here; on QNN (8,126 CX) it succeeds

TABLE VI
80-QUBIT SUITE, OWN-PROTOCOL COMPARISON (ARCHITECTURE H, SAME PHYSICAL TOPOLOGY AS THE 64-QUBIT SUITE, MORE LOGICAL QUBITS PER CIRCUIT). METHODOLOGY AND NOTATION AS TABLE IV. *TELESABRE FAILS TO CONVERGE; GMEAN IS OVER THE 2 CIRCUITS ON WHICH IT DOES. QNN HERE IS THE RECONSTRUCTED DEEP ZZ-FEATURE-MAP QNN (12,718 CX, APPENDIX A), NOT THE DEGENERATE STOCK CHAIN.

	TELESABRE		DSABRE		PORTER		
Circuit	EPR	SWAP	EPR	SWAP	EPR	SWAP	Δ_{TS}
ae*	—	—	513	3441	490	3156	—
ghz	19	172	10	93	7	115	−63.2
graphstate	88	521	25	144	28	198	−68.2
qft*	—	—	349	2630	446	2968	—
qnn*	—	—	869	13500	1517	10920	—
random*	—	—	—	—	11040	62119	—
gmean (2)	40.9		15.8		**14.0**		**−65.8**

TABLE VII
SHARED-LAYOUT COMPARISON, ALL SUITES: DSABRE AND PORTER ARE BOTH ROUTED, SINGLE-SHOT, FROM TELESABRE'S OWN INITIAL LAYOUT (SECTION IV-C). Δ_{TS} COLUMNS GIVE EACH ROUTER'S EPR CHANGE RELATIVE TO TELESABRE'S OWN-PROTOCOL RESULT; NEGATIVE IS BETTER. ROWS EXIST ONLY WHERE TELESABRE CONVERGES AND THUS PROVIDES A LAYOUT TO SHARE: ALL SIX CIRCUITS AT 25Q, FIVE OF SIX AT 64Q (RANDOM MISSING), TWO OF SIX AT 80Q (GHZ AND GRAPHSTATE); PER-SUITE GMEANS ARE OVER THE AVAILABLE ROWS.

		TELESABRE		DSABRE			PORTER		
Suite	Circuit	EPR	SWAP	EPR	SWAP	Δ_{TS}	EPR	SWAP	Δ_{TS}
25q	ae	118	370	77	393	−34.7	62	267	−47.5
25q	ghz	11	30	8	33	−27.3	6	26	−45.5
25q	graphstate	17	43	10	37	−41.2	14	57	−17.6
25q	qft	141	371	89	410	−36.9	96	343	−31.9
25q	qnn	357	770	177	818	−50.4	158	707	−55.7
25q	random	632	1512	439	1529	−30.5	412	1392	−34.8
	gmean (6)	94.3		59.1		**−37.3**	56.5		**−40.1**
64q	ae	323	1508	183	1963	−43.3	233	1753	−27.9
64q	ghz	11	111	8	124	−27.3	8	104	−27.3
64q	graphstate	76	379	32	167	−57.9	49	289	−35.5
64q	qft	410	1872	228	2104	−44.4	227	1668	−44.6
64q	qnn	1365	6292	544	8250	−60.1	679	5963	−50.3
	gmean (5)	172.1		89.7		**−47.9**	107.1		**−37.8**
80q	ghz	19	172	15	173	−21.1	11	156	−42.1
80q	graphstate	88	521	55	306	−37.5	71	402	−19.3
	gmean (2)	40.9		28.7		**−29.8**	27.9		**−31.7**

at 1,365 EPR pairs and 6,292 SWAPs, exactly matching DSABRE's own published number for this circuit [5] – a useful cross-check that our harness reproduces TELESABRE faithfully. The gmean row is computed over the five circuits on which TELESABRE converges. PORTER improves on TELESABRE on every converged circuit (−36.1% to −73.7%); notably, the chain-structured GHZ that defeats locally-greedy schedulers generally – DSABRE regresses against TELESABRE on it both in our reproduction (14 vs. 11) and in its own published table [5] – is now a PORTER *win* (6 EPR pairs vs. 11 and 14): the first-appearance visit order of the temporal locality-greedy layout places successive chain neighbours in the same core, dissolving the pathology at the layout stage rather than fighting it at routing time. Against DSABRE, PORTER takes the gmean (86.4 vs. 89.0), winning AE (177 vs. 262), GHZ, and Random (737 vs. 770, where TELESABRE completes neither); DSABRE retains Graphstate (18 vs. 20), QFT (258 vs. 262, narrowly), and, most clearly, the deep QNN (509 vs. 864). That QNN row is PORTER's weakest result in any suite and an honest limit of the single-layout policy: it is the one benchmark where the deterministic layout is decisively worse than alternatives PORTER itself can produce (its retired best-of-six search reached 642 there), suggesting layout sensitivity that a future refinement pass could recover.

80-qubit suite. TELESABRE converges on only two of the six circuits here (GHZ and Graphstate; Table VI), so its gmean basis is thin. PORTER edges that basis (14.0 vs. DSABRE's 15.8, TeleSABRE's 40.9), driven by GHZ (7 EPR pairs vs. 10 and 19). Completion is the clearer story at this scale: PORTER finishes all six circuits, DSABRE five (missing Random), TELESABRE two. Over the five circuits DSABRE also completes, however, the per-core corner reservation pays off for the baseline: PORTER wins AE (490 vs. 513) and GHZ (7 vs. 10) but loses Graphstate (28 vs. 25), QFT (446 vs. 349), and, decisively, the deep QNN (1,517 vs. 869), for a geometric mean of 145.4 against DSABRE's 131.2 – the one own-protocol basis in our evaluation on which a baseline's best-of-three beats PORTER's single run.

The deep QNN deserves emphasis because it corrects a defect in the original suite: at 80 qubits the stock `mqt.bench` QNN is a 79-CX chain *identical* to GHZ (Appendix A), which would have double-counted the easy chain case and hidden the hard one. The reconstructed 12,718-CX QNN is a fully-connected interaction graph, and on it PORTER loses to DSABRE (1,517 vs. 869) by the same mechanism as the 64q QNN (864 vs. 509): its single deterministic layout cannot match DSABRE's best-of-three SabreLayout partition on an all-to-all circuit. The two QNN rows are now a consistent, honest boundary of the single-layout policy – the regime a future layout- or route-refinement pass (Section VI) most needs to target.

On Random (23,381 CX, by far the largest circuit in any suite), PORTER is the only router to finish, in 37 s at 11,040 EPR pairs, while DSABRE exceeds the uniform 900 s per-call timeout (Section IV-B). Running DSABRE past that cap to characterize the failure shows it is not a deadlock: its `max_iterations=20000` budget (carried over from the 64-qubit suite per DSABRE's own scaling guidance [5]) is reached after 938 s of its first forward pass alone, having cleared 1,233 of the 23,381 CX gates with genuine forward progress – a resource-budget limitation on an unusually dense circuit (3–18× the CX/qubit ratio of any other benchmark), not an in-principle inability. Still, the practical cost of a budget large enough to finish (three layout candidates through up to three passes each) would run to hours against PORTER's 37 seconds.

E. Inter-Core Action Mix

Table VIII decomposes PORTER's EPR budget on every own-protocol run into the individual inter-core actions committed, making visible how the three primitives and the two relief tiers (Sections III-D and III-H) actually divide the work.

TABLE VIII
PORTER'S INTER-CORE ACTION MIX UNDER THE OWN PROTOCOL (THE SAME RUNS AS TABLES IV–VI): TELEDATA TELEPORTS BY PLAN PROVENANCE (NORMAL; BLOCKED PLAN SELECTED VIA OPPORTUNISTIC RELIEF; TAIL-RELIEF BFS), SINGLE TELEGATES, AND TELEGATEPACKETS WITH THE TOTAL NUMBER OF REMOTE CX GATES THEY EXECUTE AND THE MEAN/MAXIMUM PER PACKET. EVERY ACTION CONSUMES ONE EPR PAIR, SO THE FIVE ACTION-COUNT COLUMNS SUM TO THE EPR COLUMN.

		TELEDATA			TELEGATE	TELEGATEPACKET			
Suite	Circuit	norm.	blk.	tail		count	CX	size	EPR
25q	ae	43	14	0	0	8	50	6.2/8	65
25q	ghz	4	0	0	0	0	0	—	4
25q	graphstate	7	0	0	0	0	0	—	7
25q	qft	58	16	0	0	0	0	—	74
25q	qnn	89	30	0	0	0	0	—	119
25q	random	261	4	0	93	43	90	2.1/3	401
64q	ae	100	43	2	0	32	101	3.2/8	177
64q	ghz	6	0	0	0	0	0	—	6
64q	graphstate	20	0	0	0	0	0	—	20
64q	qft	179	37	0	1	45	105	2.3/6	262
64q	qnn	464	150	5	5	240	630	2.6/8	864
64q	random	405	16	11	228	77	157	2.0/3	737
80q	ae	147	57	7	15	264	834	3.2/8	490
80q	ghz	7	0	0	0	0	0	—	7
80q	graphstate	28	0	0	0	0	0	—	28
80q	qft	166	62	7	2	209	553	2.6/8	446
80q	qnn	778	254	7	30	448	1182	2.6/8	1517
80q	random	6559	257	114	3538	572	1209	2.1/4	11040

Primitive split. TELEDATA dominates on every circuit: moving a qubit resolves not just the seed gate but its latent and lookahead successors, so the plan scorer usually prefers it. Single TELEGATEs are almost absent from the structured circuits and concentrate on Random (93, 228, and 3,538 across the three scales) — isolated remote CX gates between qubits that share no further work, exactly the case where entangling once and moving nothing is cheapest. TELEGATEPACKETs carry substantial CX volume on the dense circuits: 80q AE executes 834 remote CX gates in 264 packets (3.2 per EPR pair), 80q QNN 1,182 in 448, and on 25q AE the mean packet reaches 6.2 remote CXs, close to the size cap of 8 (Table III), which is attained on every AE/QNN instance that forms packets at all. GHZ and Graphstate, in contrast, route on teleports alone — their chain and star interactions offer neither same-control runs nor capacity pressure — which is why they are omitted from the mechanism ablation below.

Relief tiers. Selecting a *blocked* plan via the opportunistic comparator is a regularly exercised path, not a corner case: it accounts for 14–30 committed teleports per dense 25q circuit and up to a quarter of all teleports on 80q QNN (254 of 1,039). Tail-relief BFS, the last resort when no normal plan exists at all, fires an order of magnitude less often and almost exclusively on Random (114 teleports at 80q, 11 at 64q) — consistent with its role as the safety net that keeps the deterministic-greedy loop alive at high occupancy rather than a routine routing device.

F. Shared-Layout Results

The own-protocol comparison mixes two effects: the quality of each router's initial layout and the quality of its routing decisions. Table VII removes the first effect entirely – both DSABRE and PORTER route, single-shot, from the identical layout TELESABRE built for itself – so what remains is a pure routing-quality comparison on a mapping optimised for neither router.

25-qubit suite. Under TELESABRE's own initial layout, both DSABRE and PORTER still improve substantially on TELESABRE (-37.3% and -40.1% gmean respectively), with PORTER the better of the two on four of six circuits (AE, GHZ, QNN, Random) and on the gmean. DSABRE keeps Graphstate and QFT, the circuits its partition-then-refine strategy suits best under its own protocol too.

64-qubit suite. TELESABRE's initial layout is available for five of six circuits. DSABRE wins the gmean here (-47.9% vs. PORTER's -37.8%): it takes AE, Graphstate, and QNN outright, ties GHZ (both -27.3%), and is edged out only narrowly on QFT (228 vs. 227 EPR). On architecture H's larger, 16-slot cores, DSABRE's forward–backward–forward routing passes evidently extract more from a fixed externally supplied layout than PORTER's single forward pass does – the one setting in our evaluation where they do. PORTER nonetheless wins this suite end-to-end (Table V), because its own layout is enough better than the one being shared here to more than repay the difference; a backward-refinement pass over PORTER's route is the natural mechanism to close the residual gap (Section VI).

80-qubit suite. TELESABRE's initial layout can be extracted only for the two circuits on which it converges (GHZ and Graphstate), so this block is indicative rather than conclusive. PORTER edges DSABRE on the two-circuit gmean (-31.7% vs. -29.8%), winning GHZ (11 vs. 15 EPR pairs) while DSABRE keeps Graphstate. GHZ is a win for PORTER even from a layout it did not build, so its chain-circuit strength is a property of the router as well as of its layout.

Fixed-layout comparison, pooled. A second source of externally supplied layouts — `pytket-dqc`'s `HypergraphPartitioning` placements, routed by both remaining routers on the 25q and 64q suites — is reported in Appendix B. Pooling that grid with Table VII gives thirty instances on which DSABRE and PORTER routed the *identical* externally supplied layout: PORTER uses fewer EPR pairs on fifteen, DSABRE on twelve, with three ties, and PORTER's pooled geometric mean is lower (73.8 vs. 76.7). Whatever share of each router's end-to-end result its own layout contributes, the routing stages themselves are at worst at parity – and PORTER reaches that parity in a single forward pass, without DSABRE's forward–backward–forward schedule.

G. Mechanism Ablation

Table IX attributes PORTER's EPR consumption to its two distinctive mechanisms by disabling each in isolation, holding the initial layout and every other parameter fixed (the six omitted rows – GHZ and Graphstate at every scale – form no packets and never trigger relief, Table VIII, so all three configurations coincide there).

Packets. Disabling TELEGATEPACKET forces every shared-control remote CX run to pay per gate (single telegates or

TABLE IX
MECHANISM ABLATION: PORTER EPR WITH THE TELEGATEPACKET PRIMITIVE DISABLED (MAX_PACKET_SIZE=1) AND WITH BOTH CAPACITY-RELIEF TIERS DISABLED, ALL FROM THE SAME TEMPORAL LOCALITY-GREEDY LAYOUT AT ROUTER SEED 0. "PK" IS THE PACKET COUNT OF THE FULL CONFIGURATION. †FAILS TO COMPLETE (DEADLOCK OR ITERATION-GUARD TRIP). THE GMEAN ROW IS OVER THE FOURTEEN INSTANCES ALL THREE CONFIGURATIONS COMPLETE.

Suite	Circuit	Full	No packets	No relief	pk
25q	ae	65	71	80	8
25q	qft	74	74	140	0
25q	qnn	119	122	212	0
25q	random	401	412	395	43
64q	ae	177	211	327	32
64q	qft	262	372	382	45
64q	qnn	864	931	600	240
64q	random	737	843	789	77
80q	ae	490	688	†	264
80q	qft	446	435	†	209
80q	qnn	1517	1571	†	448
80q	random	11040	11628	†	572
gmean (14)		57.6	61.4	67.0	

teleports). The cost tracks the packet count: +42% EPR on 64q QFT (262 → 372, 45 packets), +40% on 80q AE (490 → 688, 264 packets), +19% on 64q AE, +14% on 64q Random, and smaller gains on the packet-heavy deep-QNN rows (80q QNN 1,517 → 1,571, 448 packets), against a +7% overall gmean. The lone counter-move (80q QFT, 446 → 435) shows packet formation is a heuristic commitment that can occasionally displace a slightly better teleport plan, but the direction is overwhelmingly one-sided.

Capacity relief. Disabling both relief tiers (Section III-H) is not merely more expensive – it is *unsafe*: 80q AE, QFT, QNN, and Random all fail to complete, because with every candidate destination below $f_{\min}$ free slots and no mechanism to drain qubits toward reservoir cores, the deterministic-greedy loop has no legal remote plan left. Where routing does complete, dense circuits pay heavily (+89% on 25q QFT, +78% on 25q QNN, +85% on 64q AE). The one instance relief *hurts* (64q QNN, 864 → 600 without it) shows the opportunistic tier can overpay on a circuit whose congestion would resolve itself; tightening that comparator is a natural tuning opportunity. Overall, relief is what converts PORTER's raise-on-deadlock failure model (Section III-D) from a liability into a non-event: with both tiers active, no benchmark instance in any protocol deadlocks.

H. Compile Time

Table X reports per-attempt wall-clock time: one `route` call for DSABRE (one SabreLayout candidate through its forward–backward–forward pass schedule), one `telesabre_run` attempt for TELESABRE, and one `compile` call for PORTER, all on the same machine. For the baselines the own-protocol totals in Section IV-D are 3× these per-attempt figures (best-of-three); for PORTER the per-attempt figure *is* the total, since its protocol is a single run from a single deterministic layout. PORTER is consistently

TABLE X
PER-ATTEMPT WALL-CLOCK TIME (SECONDS) ON REPRESENTATIVE CIRCUITS; ONE CANDIDATE LAYOUT/SEED PER ROUTER. *DID NOT COMPLETE.

Circuit	CX	TELESABRE	DSABRE	PORTER
ae (25q)	558	0.16	0.78	0.07
random (25q)	1124	1.31	5.41	0.42
ae (64q)	1962	0.99	10.61	0.32
qft (64q)	1966	1.89	8.44	0.41
qnn (64q)	8126	21.54	135.65	2.23
random (80q)	23381	*	*	37.40

the fastest per attempt, and the gap widens with circuit size: on the 8,126-CX QNN circuit it is 61× faster than DSABRE's single attempt (182× against the best-of-three total); on the 1,962-CX AE circuit, 33×; on the 23,381-CX 80-qubit Random circuit, DSABRE's own-protocol attempt does not complete at all (Table VI) while PORTER finishes in 37 s. This reflects the performance engineering of Section IV-A as much as any algorithmic difference between the routers – DSABRE's reference implementation has not been subjected to the same optimisation pass – so it should be read as a statement about implementation maturity rather than an inherent complexity gap between the two approaches.

V. RELATED WORK

TELESABRE [6] is our closest architectural relative: both routers support teledata and gate teleportation and both build on the SABRE extended-set scoring idea. The two differ chiefly in decision structure: TELESABRE scores inter-core candidates via a contracted-communication-graph Dijkstra energy that mixes staging cost and lookahead into one flat action pool per candidate, whereas PORTER scores remote *plans* independently of their preparation cost and only generates preparation SWAPs for the one plan selected (Section III-D), and adds a same-control packet primitive that amortises runs of shared-control remote CX gates onto one EPR pair. PORTER's capacity handling is also more proactive: alongside TELESABRE's safety-valve iteration cap, PORTER maintains an explicit blocked-plan set and both a breadth-first tail-relief search and an opportunistic pre-emptive relief comparator (Section III-H).

DSABRE [5] (we compare against its default `dSABRE_BurstExt` variant, "DSABRE") is the stronger of the two baselines and the closest competitor on EPR count. Its strategy inverts PORTER's cost profile: a SabreLayout-based initial allocation refined through forward–backward–forward routing passes, run over three seeds, buys excellent core partitions – it remains the strongest router on the sparse, highly partitionable Graphstate family at 64 and 80 qubits in every protocol we test – at a compile-time cost that is 11–61× PORTER's per attempt (Section IV-H). Under each router's own protocol PORTER's single run posts the lower geometric-mean EPR on all three suites (Section IV-D), and across thirty instances routed from identical externally supplied layouts, PORTER wins fifteen to DSABRE's twelve (Section IV-F); DSABRE's remaining advantage is concentrated in its multi-pass refinement of an externally

supplied layout on large cores (Section IV-F) and on the densest own-protocol instances – the all-to-all QNN at 64q and 80q and the 80q QFT – where its best-of-three partition beats PORTER's single deterministic layout. DSABRE restricts itself to teledata; PORTER follows TELESABRE in also supporting gate teleportation and extends it with same-control packets, which matters on circuits with long runs of shared-control remote CX gates (e.g. QFT-like phase-kickback patterns) where amortising many remote CX gates onto one EPR pair can outweigh a less-optimal core partition (Section IV-G).

Single-QPU qubit mapping. The front+extended-set scoring loop PORTER and both baselines build on originates with SABRE [11]; LIGHTSABRE [14] formalises the Δ-form SWAP score PORTER reuses for its intra-core stage (Eq. 1) and inter-core direct-gain term (Eq. 3).

Allocation and scheduling. Hypergraph-partitioning allocation methods [13], [15] and time-aware schedulers [16], [17] address the upstream allocation and downstream scheduling stages respectively. Both remain complementary to PORTER: its own temporal locality-greedy allocation is a one-pass greedy heuristic, not a global optimiser, and Appendix B shows a hypergraph partition can still improve the most partitionable circuits when its generation cost is acceptable.

VI. CONCLUSION

We presented PORTER, a plan-committed router for multi-core distributed quantum computers whose central design choice is to separate remote-plan *selection* from remote-plan *preparation*, in contrast to TELESABRE and DSABRE, which score inter-core candidates and their staging SWAPs together in one flat action pool: every legal teleport, telegate, and same-control packet candidate (including capacity-blocked variants) is scored on its physical and DAG-lookahead merits before any core-preparation SWAP is generated, and preparation effort is then spent exclusively on the one plan chosen. The router is paired with a two-tier capacity-relief mechanism (breadth-first tail relief when every plan is blocked, opportunistic pre-emptive relief when one is severely blocked) and a deterministic, seed-free temporal locality-greedy initial layout that co-locates qubits by temporally discounted interaction weight in first-use order.

The combination is deliberately minimal – one layout, one forward pass, no seed sweep, no rollback – and the evaluation shows the minimalism costs nothing on this benchmark family: a single PORTER run posts the lowest geometric-mean EPR count of the three routers on all three suites (54.3%, 49.8%, and 65.8% below TELESABRE; below DSABRE's best-of-three on each suite), completes all eighteen benchmarks where DSABRE completes seventeen and TELESABRE thirteen, and runs 11–$61\times$ faster per attempt than DSABRE. Across thirty instances routed from identical externally supplied layouts, PORTER's single forward pass matches or beats DSABRE's forward–backward–forward schedule (fifteen wins to twelve). The ablation attributes the gains concretely: packets save up to 42% EPR on shared-control-heavy circuits, and capacity relief is what makes the deterministic-greedy design complete at all at 80 qubits. At the largest scale we tested (a 23,381-CX circuit), PORTER completed in 37 s where both baselines exceed a uniform 900 s per-call compiler timeout – for DSABRE a resource-budget limitation rather than a deadlock (Section IV-D), but a practical throughput gap of two orders of magnitude nonetheless. Every reported run was certified by an independent replay verifier that re-derives, from the emitted operation stream alone, gate coverage, dependency order, and physical legality.

Two limits are equally concrete. DSABRE remains the strongest router for extreme partitionable structure (Graphstate at 64 and 80 qubits, in every protocol), and its best-of-three SabreLayout partition beats PORTER's single deterministic layout on the densest instances: the deep 64q and 80q QNN rows (864 vs. 509 and 1,517 vs. 869 EPR pairs) and the 80q QFT (446 vs. 349) are PORTER's weakest results and a consistent boundary of the single-layout policy. Natural next steps follow directly: a backward-refinement pass over PORTER's single forward route, a lightweight exchange-refinement of the temporal locality-greedy partition targeting the dense-QNN case, and extending the packet primitive beyond the shared-control case.

Artefact availability.: The PORTER implementation, the independent replay verifier, the cross-router comparison harness (including the patched TELESABRE binary and its documentation), and raw per-circuit JSON results are publicly available at https://github.com/ebony72/porter. Appendices A–D (the 80-qubit benchmark reconstruction, the pytket-dqc layout comparison and opt-in study, and the replay-verification detail) are provided as a separate supplementary document.

ACKNOWLEDGMENTS

This work was partially supported by the Natural Science Foundation of China (Grant No.). The author used Anthropic's Claude (via Claude Code) to assist with: (i) reviewing and applying behaviour-preserving performance fixes to the PORTER implementation, (ii) authoring the independent replay verifier and the cross-router comparison harness, (iii) diagnosing and patching the TELESABRE reference-implementation issue described in Section IV-B, and (iv) drafting the LaTeX source of this manuscript. All AI-generated content was reviewed and verified by the author, who takes full responsibility for the correctness and originality of the work.

REFERENCES

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.
[6] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.
[7] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.
[8] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.
[9] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.
[10] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.
[11] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.
[12] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.
[13] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.
[14] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A lightweight and enhanced SABRE algorithm," 2024. [Online]. Available: https://arxiv.org/abs/2409.08368
[15] P. Andres-Martinez and C. Heunen, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.
[16] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.
[17] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.